

MOIVE DA - MERN PROJECT REPORT

Table of Contents

1. INTRODUCTION
1.1 OVERVIEW OF THE PROJECT
1.2 DESCRIPTION OF MODULES
2. SYSTEM SPECIFICATION
2.1 HARDWARE SPECIFICATION
2.2 SOFTWARE SPECIFICATION
2.3 SOFTWARE DESCRIPTION
3. SYSTEM STUDY
3.1 EXISTING SYSTEM
3.1.1 DRAWBACKS OF EXISTING SYSTEM
3.2 PROPOSED SYSTEM
3.2.1 FEATURES OF PROPOSED SYSTEM
4. SYSTEM DESIGN AND DEVELOPMENT
4.1 DATA FLOW DIAGRAM
4.2 INPUT DESIGN
4.3 OUTPUT DESIGN
5. TESTING AND IMPLEMENTATION
5.1 UNIT TESTING
5.2 SCROLL ANIMATION TESTING
5.3 RESPONSIVE DESIGN TESTING
5.4 CROSS-BROWSER COMPATIBILITY TESTING
5.5 PERFORMANCE TESTING
5.6 IMPLEMENTATION DETAILS
6. CONCLUSION
7. SCOPE FOR FUTURE ENHANCEMENT
BIBLIOGRAPHY
APPENDICES
A) SAMPLE CODING
B) REPORT (SCREENSHOTS)

1. INTRODUCTION

Moive Da is a modern full-stack web application built using the MERN architecture (MongoDB, Express.js, React, Node.js). The platform aggregates movie information and ratings, with special focus on Tamil cinema collections. The system offers rich user experience through dynamic filters, sorting, view switching, animation effects, and role-based interactions.

The project demonstrates practical implementation of:

- Full-stack API-driven architecture
- Authentication and authorization using JWT
- User-personalized lists (favorites/watchlist)
- Admin CRUD management
- Responsive, animated, and interactive frontend UI

1.1 OVERVIEW OF THE PROJECT

The system has two major layers:

- **Frontend (React + Vite):** UI rendering, user interactions, filtering, pagination, effects, authentication forms, and dashboard options.
- **Backend (Node + Express + MongoDB via Mongoose):** REST APIs, business logic, JWT auth, role checks, movie data storage, and seeded Tamil movie catalog.

The application currently supports:

- Public movie browsing
- Login/Register
- Favorite and watchlist toggles per user
- Admin actions (create/update/delete movie records)
- Real-time UI updates with toasts and loading skeletons

1.2 DESCRIPTION OF MODULES

1. **Authentication Module**

- User registration
- User login
- JWT generation and verification
- Current user profile retrieval

2. **Movie Catalog Module**

- Fetch movies with filters
- Sorting by latest, year, score
- Pagination support
- Movie detail modal display

3. **User Personalization Module**

- Favorites management
- Watchlist management
- User-specific lists API

4. **Admin Management Module**

- Create movie
- Update movie
- Delete movie
- Role-based protection using admin check

5. **UI/UX Module**

- Dark/light theme switch
- Grid/list view switch
- Skeleton loading placeholders
- Toast notifications
- Animation-enhanced hero and cards

2. SYSTEM SPECIFICATION

2.1 HARDWARE SPECIFICATION

Minimum:

- Processor: Intel i3 / AMD equivalent

- RAM: 4 GB
- Storage: 2 GB free disk space
- Internet: Required initially for package setup and in-memory Mongo binary download

Recommended:

- Processor: Intel i5/i7
- RAM: 8 GB or above
- SSD storage for faster builds

2.2 SOFTWARE SPECIFICATION

- Operating System: Windows 10/11
- Runtime: Node.js-compatible runtime (Bun used in this setup)
- Package manager: Bun / npm
- Frontend build tool: Vite
- Backend framework: Express.js
- Database: MongoDB (with in-memory fallback through mongodb-memory-server)
- IDE: Cursor / VS Code
- Browser: Chrome, Edge, Firefox

2.3 SOFTWARE DESCRIPTION

- **React:** Component-driven frontend rendering
- **Vite:** Fast development server and HMR support
- **Express.js:** Lightweight API routing and middleware
- **Mongoose:** MongoDB ODM for schema and model handling
- **JWT:** Stateless user session security
- **Bcryptjs:** Password hashing for secure storage

3. SYSTEM STUDY

3.1 EXISTING SYSTEM

Typical existing movie listing sites provide static catalogs or basic list pages with limited personalization and no admin control for custom data entry in small educational/demo environments.

3.1.1 DRAWBACKS OF EXISTING SYSTEM

- Limited interactivity and customization
- No integrated favorites/watchlist for demo users
- Weak admin tooling in basic sample apps
- Poor UI effects and user engagement
- Limited Tamil movie-focused curated collection

3.2 PROPOSED SYSTEM

The proposed system (Moive Da) is a complete MERN web app with enhanced frontend effects, JWT authentication, role-based admin actions, and user personalization features. The design is modular and scalable.

3.2.1 FEATURES OF PROPOSED SYSTEM

- JWT-based login/register
- Role-based admin CRUD
- Favorites/watchlist per user
- Rich filter + sorting + pagination
- Loading skeletons and toast notifications
- Theme and layout toggles
- 20+ seeded Tamil movie entries

4. SYSTEM DESIGN AND DEVELOPMENT

4.1 DATA FLOW DIAGRAM

High-level flow:

1. User opens frontend
2. Frontend calls backend APIs
3. Backend validates request and queries MongoDB
4. Backend returns JSON response
5. Frontend renders data and interactions

Auth flow:

1. User submits register/login form
2. Backend validates and issues JWT
3. Frontend stores token
4. Token attached in protected requests
5. Backend verifies token and role

4.2 INPUT DESIGN

Key input screens:

- Search bar (q)
- Genre dropdown (genre)
- Minimum score slider (min)
- Sort dropdown (sort)
- Login/Register form inputs
- Admin movie form (title, year, genres, language, runtime, ratings)

Input validation:

- Required checks for auth fields
- Password minimum length
- Numeric validation for rating/year/runtime
- Backend authorization checks for protected operations

4.3 OUTPUT DESIGN

Output screens/components:

- Movie cards with poster, title, score, metadata

- Modal detail view on card click
- Favorite/watchlist toggle states
- Toast notification messages
- Skeleton loading placeholders
- Paginated results with page controls

5. TESTING AND IMPLEMENTATION

5.1 UNIT TESTING

Suggested unit test coverage:

- Auth payload validation
- Token verification middleware
- Movie score aggregation logic
- Favorites/watchlist toggle logic

5.2 SCROLL ANIMATION TESTING

Testing checks:

- Hero animation loads once without jitter
- Card hover and transition remain smooth
- No animation overlap causing layout shift

5.3 RESPONSIVE DESIGN TESTING

Validated behavior:

- Grid adapts to viewport size
- Controls remain accessible on smaller widths
- Modal and toast positions remain usable

5.4 CROSS-BROWSER COMPATIBILITY TESTING

Tested in:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Validation points:

- Theme switch behavior
- Form submission flow
- API data rendering consistency

5.5 PERFORMANCE TESTING

Checks performed:

- API response under filtered queries
- Pagination reduces payload size per request
- Skeleton loading improves perceived responsiveness
- Vite HMR updates quickly in development

5.6 IMPLEMENTATION DETAILS

Project path:

- D:/movie-agg-run/mern

Backend:

- server/src/index.ts

Frontend:

- client/src/App.tsx
- client/src/styles.css

Core endpoints:

- GET /api/health
- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/me
- GET /api/movies
- POST /api/users/favorites/:movieId
- POST /api/users/watchlist/:movieId
- GET /api/users/lists
- POST/PUT/DELETE /api/admin/movies

6. CONCLUSION

The Moive Da project successfully delivers a complete MERN movie platform with:

- Functional backend APIs
- Secure authentication workflow
- Personalized user modules
- Admin data management
- Strong modern UI/UX with effects and responsive behavior

The application is suitable for academic demonstration, portfolio presentation, and further extension into production-scale architecture.

7. SCOPE FOR FUTURE ENHANCEMENT

- Persistent external MongoDB deployment configuration
- Advanced role management (editor/moderator)
- Comment/review and rating submission by users
- Recommendation engine based on watchlist/favorites
- Multi-language subtitles and localization
- Image upload integration for posters
- Analytics dashboard for admin

BIBLIOGRAPHY

1. React Documentation - <https://react.dev>
2. Vite Documentation - <https://vitejs.dev>
3. Express Documentation - <https://expressjs.com>
4. Mongoose Documentation - <https://mongoosejs.com>
5. JWT Introduction - <https://jwt.io/introduction>

APPENDICES

A) SAMPLE CODING

Sample backend route snippet:

```
ts
app.post("/api/auth/login", async (req, res) => {
  const email = String(req.body?.email ?? "").trim().toLowerCase();
  const password = String(req.body?.password ?? "");
  const user = await User.findOne({ email });
  if (!user) return res.status(401).json({ error: "Invalid credentials" });
  const ok = await bcrypt.compare(password, user.passwordHash);
  if (!ok) return res.status(401).json({ error: "Invalid credentials" });
  const token = jwt.sign({ sub: String(user._id), role: user.role }, JWT_SECRET, { expiresIn: "7d" });
  return res.json({ token, user: sanitizeUser(user) });
});
```

B) REPORT (SCREENSHOTS)

Recommended screenshot list to attach:

1. Home page with filters and movie cards
2. Login/Register panel
3. Favorites/Watchlist buttons updated
4. Admin CRUD panel visible for admin user
5. Theme switch (dark/light)
6. Pagination controls and skeleton loading
7. Toast notification sample